two popular syntax branches for the x86 assembly language — the AT&T syntax popular with Linux, and the Intel syntax popular with the Microsoft Windows operating system. Both the assemblers used in Microsoft Windows — MASM (Microsoft Macro Assembler) and TASM (Turbo Assembler) — use the Intel syntax.

First, let us discuss the AT&T syntax of the x86 assembly language. This syntax is popularised by GAS (the GNU assembler). This is an important assembler because, by default, it is used as the backend for the GNU compiler collection (GCC). Now let us see an example for an assembly language program written in AT&T syntax. We will use the GNU assembler called with the command as to assemble this program. Consider the assembly language program *atandt.asm* in AT&T syntax shown below:

```
//This is a comment in AT&T Syntax
movl $111, %eax
movl $222, %ebx
addl %ebx, %eax
addl $333, %ebx
```

Notice that the above assembly language program is minimal and will not produce any visual output. The constant value 111 is stored in the register *%eax* and the constant value 222 is stored in the register *%ebx*. It will add the two numbers in the registers *%eax* and *%ebx*, store the result in the register *%eax*, and then add the constant value of 333 to the content of the register *%ebx* and store the result in the register *%ebx* itself. The GAS assembler will assemble this program without any errors or warnings.

Now let us discuss the Intel syntax of x86 assembly language. Although it is possible to use the GNU assembler GAS itself to process assembly code in Intel syntax also, it is easier to use another assembler called Netwide Assembler (NASM). NASM is very popular with those assembly language programmers who work with both Linux and Microsoft Windows operating systems. Now let us see a program similar to the program *atandt.asm* called *intel.asm*, but written in the Intel syntax, so that we can compare the two syntax branches. The program *intel.asm* is shown below:

```
;This is a comment in Intel Syntax
mov eax, 111
mov ebx, 222
add eax, ebx
add ebx, 333
```

This program works similar to the program *atandt.asm*. The constant value 111 is stored in the register *eax* and the constant value 222 is stored in the register *ebx*. The values in the registers *eax* and *ebx* are added, and the result is stored in the register *eax*. The constant value 333 is added to the register *ebx* and the result is stored in the register *ebx* itself. Figure 3 shows the two programs being assembled with the assemblers GAS and NASM.

```
File  Edit  View  Search  Terminal  Help
% as atandt.asm
% nasm -f elf64 intel.asm
%
```

Figure 3: Assembly of the two x86 programs

As you can observe, there are a number of differences between the AT&T and Intel syntax, with the most annoying being the order of the source and destination operands. I hope the above program shows us this difference. We can see that in the AT&T syntax, the destination register comes at the end after all the source registers; whereas in the Intel syntax, the destination register comes at the beginning before any source registers. The program shows us how comments are written in different styles in each syntax. It also shows us how register names and immediate values are written in each syntax. In the AT&T syntax, register names are preceded with a percentage symbol (%). Similarly, in the AT&T syntax, immediate operands are preceded with a dollar symbol ($). The commands *addq* and *add* tell us that the two syntax branches use different techniques to identify the size of memory operands. Of course, there are many other differences between the two syntax branches, but the aim of the article is not to list all such subtle differences. Rather, the article tries to point out the variety available, while choosing an assembly language to learn.

Depending on the architecture or the operating system you choose, there is a wide variety of assembly languages, each with a different syntax to choose from. So before proceeding any further with assembly languages, I request you set your priorities regarding the architecture and operating system you plan to work with in the long term. END

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. The author maintains a technical blog at *www.computingforbeginners.blogspot.in*.